

4.Round Robin CPU Scheduling Algorithm

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where
• The first integer denotes the Arrival Time (AT)
• The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

3
2
0 5
1 3
2 1

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 struct Process {
4     int id;
5     int at;
6     int bt;
7     int ct;
8     int tat;
9     int wt;
10    int rt;
11 };
12
13 int main() {
14     int n, tq;
15     printf("Enter number of processes: \n");
16     scanf("%d", &n);
17
18     printf("Enter Time Quantum: \n");
19     scanf("%d", &tq);
20
21     struct Process p[n];
22     for (int i = 0; i < n; i++) {
23         p[i].id = i + 1;
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where
• The first integer denotes the Arrival Time (AT)
• The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

3
2
0 5
1 3
2 1

Select Language: C (GCC 9.2.0)

```
26     p[i].rt = p[i].bt;
27     p[i].ct = p[i].tat + p[i].wt + 0;
28 }
29
30 int time = 0, done = 0;
31 while (done < n) {
32     int executed = 0;
33     for (int i = 0; i < n; i++) {
34         if (p[i].rt > 0 && p[i].at <= time) {
35             executed = 1;
36             if (p[i].rt > tq) {
37                 time += tq;
38                 p[i].rt -= tq;
39             } else {
40                 time += p[i].rt;
41                 p[i].rt = 0;
42                 p[i].ct = time;
43                 done++;
44             }
45         }
46     }
47     if (!executed) {
48         time++;
49     }
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

Back To Practice

Save

Completed

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 5
1 3
2 1
```

Select Language: C (GCC 9.2.0)

```
43     p[i].ct = time;
44         done++;
45     }
46 }
47 if (!executed) {
48     time++;
49 }
50 }
51
52 float totalTAT = 0, totalWT = 0;
53 for (int i = 0; i < n; i++) {
54     p[i].tat = p[i].ct - p[i].at;
55     p[i].wt = p[i].tat - p[i].bt;
56     totalTAT += p[i].tat;
57     totalWT += p[i].wt;
58 }
59
60 printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);
61 printf("Average Waiting Time: %.2f\n", totalWT / n);
62
63 return 0;
64 }
```

Test Cases

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2